제 3 강 · 실습 2

묻고, 합의하고, 검증한다

프롬프트 작성법 × 공정표로 관리하기

건축공학전공

강의자료

03

오늘의 흐름

계산은 AI가, 읽기와 판단은 사람이.

A 프롬프트 작성법 → 컨텍스트·하네스·루프·온톨로지 → 실제 작업 흐름 7단계

B 공정표로 관리하기 — 간트·마일스톤·네트워크·주공정·진도

복습

2주차 되짚기 — 그 696자 프롬프트는 왜 통했나

목적

가상의 지상 15층 아파트 신축현장의 공정·인력·원가 데이터를 만들고, 3D 진도 대시보드를 만들어줘.

순서

① 계획을 plan.md에 10줄 이내로

② schedule.csv 생성

③ build_data.py로 data.js 변환

④ dashboard.html …

형식

열: 공종,층,시작월,종료월,인원,원가 /

40행 내외 / 단일 파일, 다크 테마, 한국어 …

제약

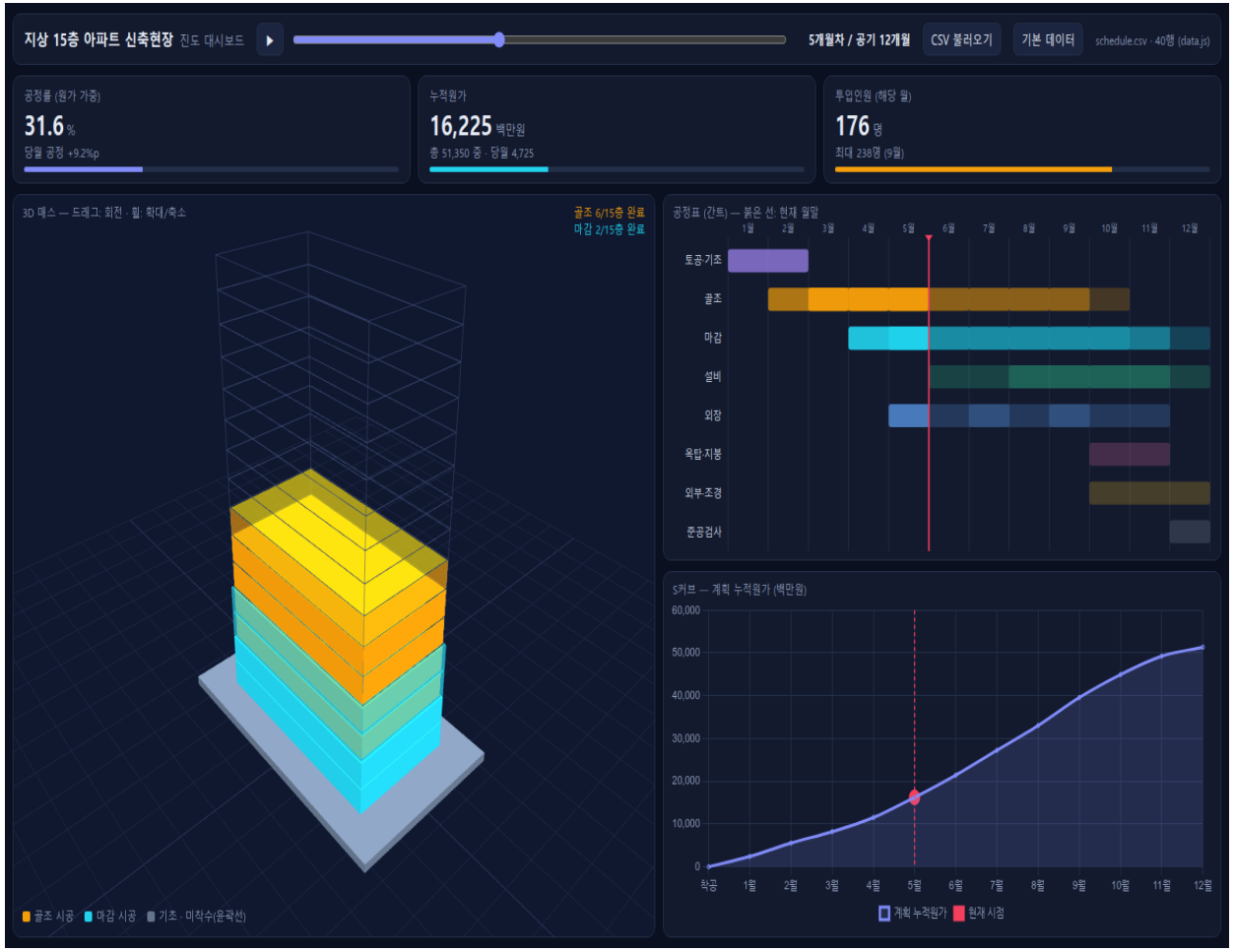
라이브러리 설치·인터넷 전송 없이,

파일은 00_practice 안에만 …

마무리

끝나면 여는 방법과 csv 수정→반영 방법을 3줄로 정리

2주차 실습 결과 — 3D 진도 대시보드



핵심 다섯 요소(목적·순서·형식·제약·마무리)를 처음부터 다 쓸 필요는 없다 → 오늘은 대화로 채운다

Part A

프롬프트 작성법

AI에게 일을 잘 시키는 말하기의 기본

A 프롬프트 엔지니어링이란

AI가 최상의 결과를 생성하도록 명령어(프롬프트)를 체계적으로 구성하고 설계하는 기술

왜 중요한가

- AI는 도구 — 정확히 쓰려면 명확한 지시(Good Input)가 필수
- 쓰레기를 넣으면 쓰레기가 나온다 (Garbage In, Garbage Out)

현장 지시와 같다

- ✗ "여기 좀 치워줘"
- ○ "A-3 공구함에 있는 M16 볼트 10개를 B-2 현장으로 옮겨주세요"

핵심 무엇을 · 어디서 · 얼마나 · 어떤 형태로 — 사람에게 지시하듯 구체적으로

A 좋은 프롬프트 vs 나쁜 프롬프트

나쁜 프롬프트

- "건설사고에 대해 써줘"
- 어떤 내용? 어떤 형식? 어느 수준?
- → AI가 추측 → 할루시네이션 가능성 ↑

좋은 프롬프트의 원칙 C.C.C.R

맥락 Context	고층 빌딩 신축 현장에서,
역할 Role	당신은 15년 경력의 안전관리 전문가입니다.
명령 Command	가장 빈번하게 발생하는 5가지 중대형 사고 유형과
결과 Result	각 유형별 즉시 적용 가능한 1차 예방 대책을 표로 만들어주세요.

합치면

고층 빌딩 신축 현장에서, 당신은 15년 경력의 안전관리 전문가입니다. 가장 빈번하게 발생하는 5가지 중대형 사고 유형과 각 유형별 즉시 적용 가능한 1차 예방 대책을 표로 만들어주세요.

A 좋은 프롬프트의 다섯 요소

역할	맥락	지시	형식	검증 기준
Role	Context	Instruction	Format	Check
누구로서 답할까	어떤 상황인가	무엇을 할까	어떤 모양으로	무엇으로 확인할까
"15년 경력 안전관리 전문가로서"	"지하2층·지상5층 도심 현장, 지금 골조 중"	"빈발 사고 5가지와 즉시 대책을"	"표로, 유형·원인·대책 3열"	"대책마다 근거 법조항, 없으면 [확인필요]"

핵심 검증 기준까지 주면 AI는 '답'을 내는 데서 멈추지 않고 '확인하고 고치는' 데까지 간다

A 구조화된 프롬프트 — 칸을 나눠 쓰기

[Context]
우리 회사는 30층 주상복합을 설계 중이며,
클라이언트가 비용 절감을 요청한 상황입니다.

[Instruction]
15년 경력의 건축 설계 전문가로서 평면도를
검토하여 비용 절감 방안을 제안하세요.

[Input Data]
연면적 15,000㎡ / RC 라멘조 / 예상 공사비 200억원
목표 절감률 10%

[Output Format]
| 항목 | 현재 사양 | 변경 제안 | 절감 효과 |

[제약 조건]
구조 안전성 타협 불가 · 건축법 준수 · A4 1장

구성요소

- ① 명령 Instruction — 할 일
- ② 맥락 Context — 배경
- ③ 입력 Input Data — 처리할 데이터
- ④ 출력 형식 Output Format — 결과 구조
- + 제약 조건 — 넘으면 안 되는 선

쓰는 요령

- 명령형 "분석하세요" · 질문형 "어떤 문제가 있나요?" · 역할형 "전문가로서 조언하세요"
- 근접성: 관련 정보는 가까이
- 구체성: 모호한 표현 지양
- 제약: 경계를 분명히

A

프롬프트 기법의 흐름



학문적 의의

■ 기계학습에서 '입력 설계'의 중요성 재조명 · 인간-AI 상호작용 방식의 전환 · 도메인 전문성과 AI의 융합

A Zero-shot · Few-shot · Many-shot — 예시를 얼마나 줄까

Zero-shot

- 예시 없이 설명만으로 수행
- 장점: 빠르고 간단
- 단점: 복잡한 작업에 한계

Few-shot

- 1~5개 예시로 패턴 제시
- 장점: 형식 일관·정확도 ↑
- 단점: 프롬프트가 길어짐

Many-shot

- 수십~수백 개 예시
- 복잡한 전문 업무,
  높은 정확도가 필요할 때

"건축 도면의 문제점을 찾아주세요."

"다음 예시와 같이 도면을 분석해주세요:
예1) 평면도: 피난통로 폭 부족 (1.2m → 1.5m 필요)
예2) 단면도: 천장고 기준 미달 (2.1m → 2.3m 필요)
[새로운 도면 분석]"

지금 에이전트에서는: 예시 양식 파일(보고서 샘플, 표 양식)을 폴더에 넣어 두고 "이 양식대로" — 이것이 few-shot

A Chain-of-Thought — 단계를 정해 생각시키기

기본 CoT

- 정의: 해결 단계를 명시해 AI가 순차적으로 추론하도록 유도
- 장점: 체계적·논리적, 단계별 검증 가능, 일관된 품질
- 단점: 단계 설정에 시간과 전문지식 필요, 유연성 부족

20층 오피스텔의 외벽 균열 발생 시 구조 안전성을
다음 단계로 평가해주세요:

1단계: 균열 현황 조사 및 분류
  – 위치·크기·방향 / 구조적·비구조적 분류
2단계: 원인 분석
  – 구조적 요인(하중, 지반침하)
  – 환경적 요인(온도변화, 건조수축)
3단계: 위험도 평가
  – 구조 안전성 영향 / 즉시보수·관찰·정밀진단
4단계: 대응방안 수립
  – 단기 안전 조치 / 장기 보수 계획 / 모니터링

각 단계별로 구체적인 분석 결과를 제시해주세요.

A Zero-shot CoT · 자기일관성(Self-Consistency)

Zero-shot CoT

- 단계를 안 주고 "단계별로 생각해 보세요" 한 줄
- 간단·빠름 / 단계 누락·일관성 부족 가능

자기일관성 Self-Consistency

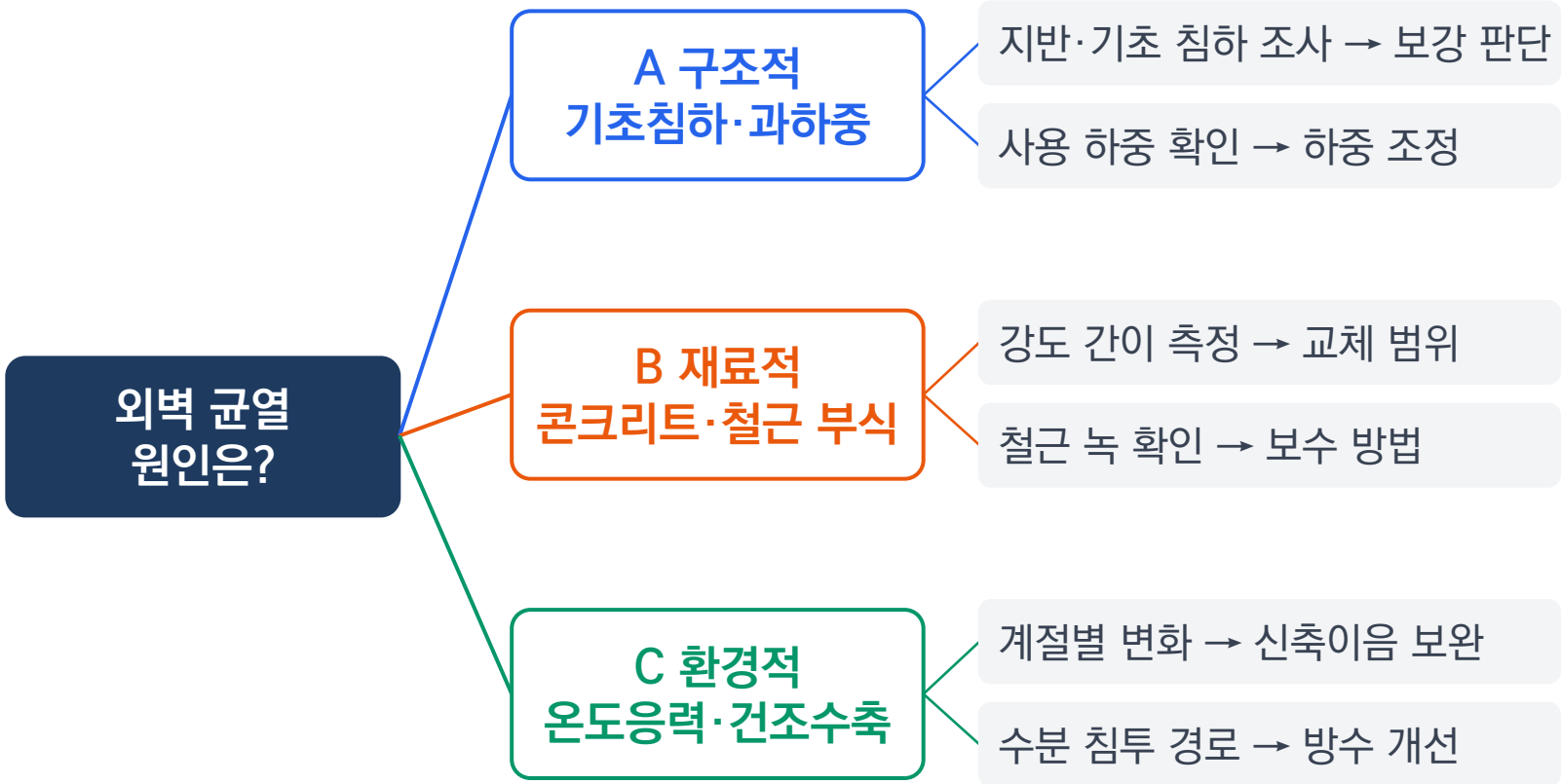
- 같은 문제를 다른 접근으로 풀고 일관된 결론 도출
- 신뢰도 ↑ ·오류 ↓ / 시간·정리 부담 ↑

```
20층 오피스텔 외벽에 균열이 발견되었습니다.
구조 안전성을 어떻게 평가해야 할지
단계별로 생각해보세요.
(Let's think step by step.)
```

```
외벽 균열 문제를 세 가지 관점에서 분석해주세요:
 1) 구조공학: 하중 경로·부재 응력·안전율
 2) 시공품질: 시공 과정·재료·양생 조건
 3) 유지관리: 경과 시간·보수 이력·환경 변화
각 관점 분석 후 세 결과를 종합하여
가장 신뢰할 수 있는 평가와 대응 방안을 제시해주세요.
```

지금 지금도 쓰는 법: 중요한 판단은 "세 관점에서 따로 보고 종합해" — 오늘 실습의 '검토자 에이전트'로 이어진다

A Tree-of-Thought — 가설을 가지 치며 비교하기



평가 기준

■ 확인의 용이성 ■ 비용 효율성 ■ 효과성

→ 가장 가능성 높은 원인부터 대응

A 메타 프롬프팅 — AI가 질문을 만들게 하기

정의: AI가 스스로 적절한 질문을 생성하고 답하도록 해, 더 포괄적이고 체계적인 분석을 끌어내는 기법

당신은 건축 전문가입니다.
"소규모 근린생활시설 리모델링 프로젝트"에 대해
가장 적절한 질문을 3가지 만들고,
각 질문에 대한 답변을 제시한 후,
종합적인 결론을 도출해주세요.

AI가 만든 질문 (예)

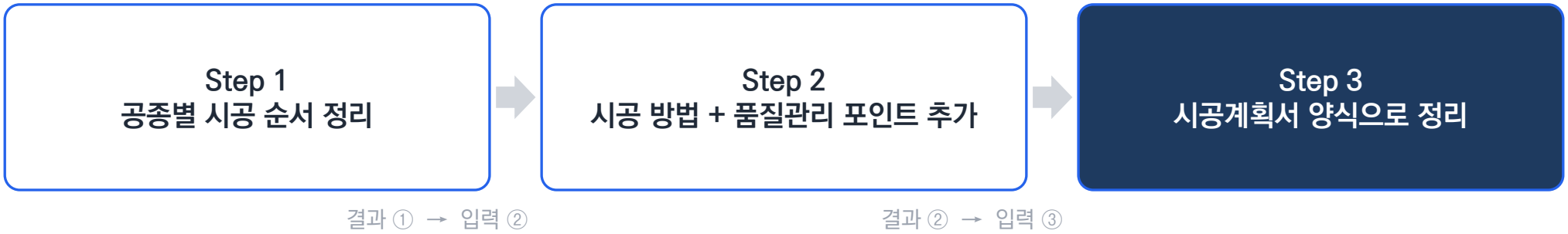
1. 주요 목표와 대상 고객은? 어느 부분을 집중 개선할까
2. 예산 안에서 법적 제약(건폐율·용적률)과 구조 보강을 어떻게 풀까
3. 준공 후 유지보수·운영 효율을 높일 핵심 전략은

지금 모를 때는 거꾸로: "이 일을 하려면 내가 무엇을 확인해야 할지 질문 5개를 만들어 줘"

A 프롬프트 체이닝 — 단계로 나눠 이어 붙이기

정의: 복잡한 작업을 여러 단계로 분할 → 각 단계의 결과를 다음 단계의 입력으로 → 순차 처리

예: 시공계획서 작성



업무별로 쓰는 예

- 업무 문서: "시공관리자로서 공사 일보를 작성하세요 [현장 정보 포함]" · 법규 검토: "산업안전보건법 기반 위험성평가 항목을 체크리스트로" · 데이터 분석: "콘크리트 강도 시험 추이를 분석하고 기준치 초과 표시"

핵심 한 번에 요청하지 말고 단계별로 나누면 훨씬 정교한 결과를 얻는다

A 자기반성 프롬프팅 — 스스로 검토하고 고치게

① 계획 수립
20층 오피스 화재안전 계획

② 체크리스트로
자체 검토

③ 문제 발견 시
수정해 최종안

검토 체크리스트:
1) 건축법 조항 인용이 정확한가? (조항 번호 확인)
2) 실무적으로 적용 가능한 구체적 방안인가?
3) 빠뜨린 중요한 화재안전 요소는 없는가?
4) 비용과 효과성을 균형 있게 고려했는가?
5) 시공성과 유지관리 측면도 충분히 검토했는가?

검토 결과 문제가 발견되면 해당 부분을 수정하여
최종 개선된 화재안전 계획을 다시 제시하세요.

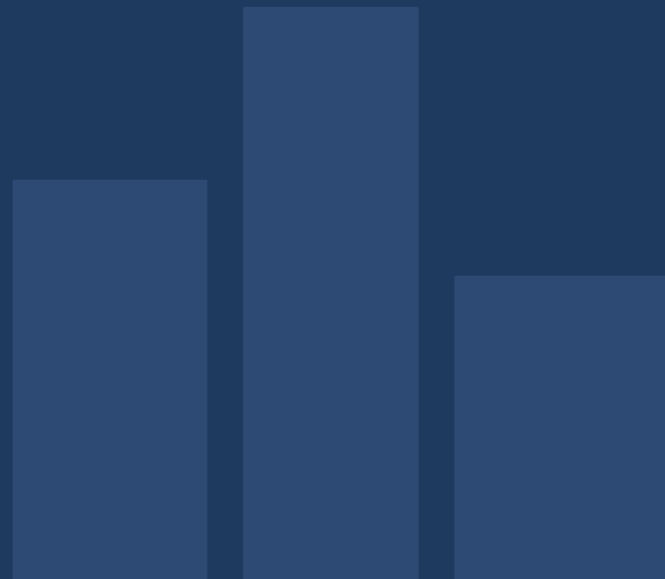
이 기법이 자라서

- 체크리스트 → 검토기준.md
- "다시 제시하세요" → 통과할 때까지 반복 (루프)
- 말로 된 검토 → 실행되는 검증기

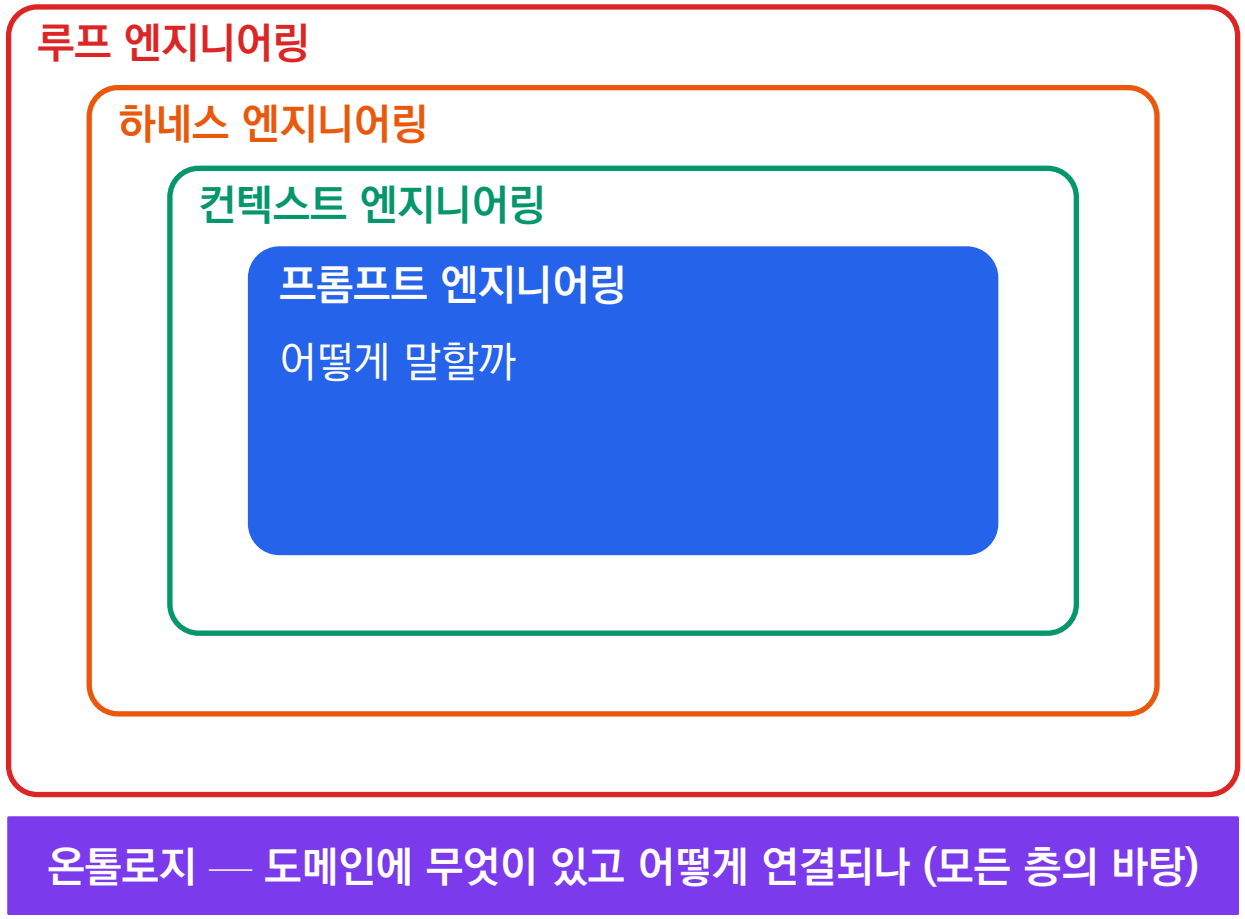
Part A

프롬프트에서 루프까지

에이전트 시대에 새로 생긴 개념들



A 프롬프트 → 컨텍스트 → 하네스 → 루프, 그리고 온톨로지



오늘 실습에서는

프롬프트
짧은 질문·지시

컨텍스트
sources 데이터팩 · HANDOFF.md · 대화 정리

하네스
폴더 규칙 · 예시 양식 · 검토기준 · 검증기 · 권한

루프
"검증기 통과까지 반복, 최대 5회"

온톨로지
작업 · 선후관계 · 마일스톤 데이터 구조

A 컨텍스트 엔지니어링 — 무엇을 보여 줄까

"추론 중에 최적의 토큰(정보) 집합을 고르고 유지하는 전략" — 프롬프트 엔지니어링의 자연스러운 다음 단계

Anthropic Engineering, 2025-09-29

왜 필요한가

- 정보가 많아질수록 기억 정확도가 떨어진다 (context rot)
- AI의 주의력에도 예산이 있다
- → 많이 넣기가 아니라 잘 골라 넣기

기법	뜻	오늘 실습에서
압축	긴 대화를 요약해 핵심만 남김	/compact
구조화된 메모	진행 상황을 바깥 파일에 기록	HANDOFF.md
하위 에이전트	일을 나눠 깨끗한 맥락에서 처리	검토자 서브에이전트
필요할 때 불러오기	처음부터 다 넣지 않고 그때그때 읽음	sources 파일을 단계마다 읽기

A 하네스 엔지니어링 — 어떤 틀에서 일하게 할까

"Humans steer. Agents execute." 사람은 방향을 잡고, 에이전트는 실행한다

OpenAI, Harness engineering, 2026-02-11

하네스 = 말의 마구

- 말을 더 세게 만드는 게 아니라, 힘을 올바른 방향으로 쓰게 하는 장치
- 에이전트가 믿고 일할 수 있는 환경 · 제약 · 피드백 장치를 설계
- 사람의 일: 직접 쓰기 → 환경 설계 · 의도 명시 · 피드백 만들기

오늘 실습의 하네스

환경	작업 폴더 규칙 · CLAUDE.md (2주차)
의도	예시 양식 · README의 계산 규약
제약	sources 수정 금지 · 권한 승인
피드백	검토기준.md + 검증기 check_schedule.py

A 루프 엔지니어링 — 언제까지 고치고, 언제 멈출까

루프 = 멈춤 조건을 만족할 때까지 일의 주기를 반복하는 에이전트

Claude Blog, Getting started with loops, 2026-06-30



루프의 종류

- 대화형: 사람이 한 턴씩
- 목표형: 통과 기준에 도달할 때까지
- 시간형: 정해진 주기마다 · 능동형: 사건이 생기면

오늘 실습의 루프 (목표형)

- 검증자 = 검토기준 + 검증기
- 멈춤 = 전부 [통과] 또는 최대 5회
- 상태 = report.md 회차별 점검표
- 새 맥락의 검토자(서브에이전트)가 덜 치우친다

A 온톨로지 — AI에게 세상의 구조를 알려주기

어떤 분야에 무엇이 있고(개념) · 어떻게 연결되며(관계) · 무엇을 할 수 있는지(행동)를 명시적으로 정해 둔 구조

객체 Object — 무엇이 있나	링크 Link — 어떻게 연결되나	액션 Action — 무엇을 할 수 있나
작업 148개 (activities.csv) 마일스톤 9개	선후관계 272개 (relations.csv) FS(끝→시작) · SS(시작→시작) · 시차	진도 반영 (--progress) 공기단축 (--crash)

오늘 데이터팩도 작은 온톨로지

■ 기업 AI: 팔란티어 온톨로지 = 객체·링크·액션 유형 위에서 에이전트가 판단·실행 | 건설: BIM(IFC)이 건물 요소의 온톨로지

핵심 데이터를 '구조'로 주면 AI가 덜 헤맨다 — 표 한 장보다 개념·관계·행동이 정해진 데이터

Part A

실제 작업 흐름

채팅으로 보는 7단계 — 오늘 실습을 미리 보기

용어

오늘 나오는 공정·현장 용어 — 모르면 이 표로

공정 용어

용어	뜻
WBS	Work Breakdown Structure. 공사를 작업 단위로 쪼갠 목차
간트차트 · ◆ 마일스톤	가로 막대 일정표 · 꼭 지켜야 할 중간 약속 날짜
FS (Finish→Start)	앞 작업이 끝나야 뒤 작업 시작 (실선)
SS (Start→Start)	앞 작업이 시작하고 시차만큼 뒤 시작 — 겹쳐 진행 (점선)
시차 (lag)	관계 사이 기다리는 날. 달력일 — 양생은 휴일에도 흐름
STD · WX 달력	STD(표준) = 토·일·공휴일 휴무 WX(기상) = STD + 비·폭염 등 기상 휴무 (토공·타설·외부)
주공정 ★ (빨강)	준공일을 결정하는 작업 줄기. 여기가 늦으면 준공이 늦어질 수 있다
여유	그 작업을 늦춰도 준공이 안 늦어지는 날 수
baseline · 진도	baseline = 처음 확정한 계획 일정 (baseline_schedule.csv) · 진도 = 실제 시작·끝난 날과 진척률 (9/30 기준)
공기단축 (crash)	사람·장비를 더 넣어 작업 기간을 줄이기 — 돈이 든다
지체상금 · 간접비	계약 준공을 넘긴 하루마다 525만원 · 현장 유지비 하루 120만원

현장 용어 (오늘 현장: 지하2·지상5층 근생)

용어	뜻
흙막이 · CIP	땅을 팔 때 주변 흙이 무너지지 않게 막는 벽 · CIP(Cast-In-Place pile) = 현장에서 콘크리트를 부어 만든 말뚝을 줄지어 세운 벽
어스앵커 · 캠빔	흙막이벽을 뒤쪽 땅에 묶어 잡는 강선 · 말뚝 머리를 한 줄로 잇는 보
터파기	지하층 깊이까지 땅을 파내는 것
매트기초	건물 바닥 전체를 한 판으로 받치는 두꺼운 기초
합벽	흙막이벽에 바로 붙여 지하 외벽을 치는 방식 — 바깥 작업 공간이 없다
거푸집 · 철근 · 타설 · 양생	콘크리트 틀 · 뼈대 보강 · 콘크리트 붓기 · 굳을 때까지 기다리기
검측	타설 전 철근·거푸집을 감리가 검사
골조 · 상량	기둥·보·슬래브 뼈대 · 골조가 맨 위층까지 끝남
비계	외벽 작업용 임시 발판 — 외부 마감 뒤 해체

읽는 법 막대 = 작업 기간 · 화살표 = 순서 · ◆ = 약속 날짜 · 빨강 = 준공을 결정하는 길 (모르면 "현장 말로 설명해 줘")

A 데이터 형식 지도 — 작업은 AI 형식으로, 납품은 사람 형식으로

AI가 작업하는 형식

- md — 문서·기준·기록
- csv — 표 (엑셀로도 열림)
- json — 구조 데이터
- html·svg — 시각화 (브라우저)
- py — 계산·검증 스크립트

변환

사람이 받는 형식

- hwpx·docx — 보고서
- xlsx — 집계표
- pptx — 발표
- pdf — 배포본
- dwg·dxf — 도면

핵심 오늘: csv·json 데이터팩 → schedule.html 공정표 → (5주차) pptx·hwpx 보고

A 데이터 형식의 실제 모습 — 열어 보면 전부 '글자'

01_검토기준.md

```
# 3주차 검토기준 — 간트·마일스톤 공정표와 세부 네트워크

> **AI가 스스로 검토할 때 쓰는 기준**입니다. 실습 ④단…
> 최종 판단은 학생이 합니다(D). 기준을 통과했어도 현장 …

## A. 자동 검증 (검증기) — 반드시 실제로 실행
- A1. 공정표에 그린 **세부작업 일정**을 `outputs/일정_…
- A2. 공정표에 표시한 **준공일·마일스톤 9개 날짜**가 `p…
- A3. 주공정(빨강)으로 칠한 작업 목록이 `baseline_sched…
```

md — #은 제목, **는 강조. 사람도 AI도 그대로 읽는다

relations.csv

```
선행,후행,관계,시차,비고
1.1.1,1.1.2,FS,0,
1.1.1,1.1.3,FS,0,
1.1.2,1.1.4,SS,3,
1.1.2,1.1.5,FS,0,
1.1.2,2.1.1,FS,0,
2.1.1,2.1.2,FS,0,
1.1.4,2.1.2,FS,0,
1.1.5,2.1.2,FS,0,
```

csv — 쉼표가 칸 구분. 엑셀로 열면 표가 된다

project.json

```
{
  "공사명": "창원 ○○동 근린생활시설 신축공사(교육용 가…
  "위치": "경남 창원시 의창구(도심지, 2면 도로·1면 인접 4…
  "규모": {
    "층수": "지하2층·지상5층·옥탑",
    "대지면적_㎡": 1000,
    "연면적_㎡": 4200,
    "지하층_㎡": "750×2",
    "지상층_㎡": "540×5"
```

json — "이름": 값 쌍, { }로 묶어 계층을 만든다

schedule.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-widt…
<title>공정표 - 창원 ○○동 근린생활시설 신축공사</title>
<style>
  :root { --crit:#d62728; --bar:#8aa4c4; --bar1:#3d5a8…
  * { box-sizing: border-box; }
```

html — 〈태그〉로 화면 구성. 브라우저로 열면 공정표

schedule.html 속 <svg> — 4.4 네트워크 그림

```
<svg id="netsvg" viewBox="0 0 1324 176" width="13…
<path class="edge" data-from="4.4.1" data-to="4.4.2…
<title>4.4.1 → 4.4.2 SS+2일</title>
<path class="edge" data-from="4.4.2" data-to="4.4.3…
<title>4.4.2 → 4.4.3 FS</title>
<rect x="14" y="70" width="200" height="92" rx="5"…
<text x="22" y="86" font-size="11.5" font-weight="7…
<text x="22" y="101" font-size="11" fill="#1f2933">2…
…
```

svg — 상자(rect)·화살표(path)·글자(text)를 좌표로 그린 그림

check_schedule.py

```
    if pg and pg.get("AS"):
        ES[c] = pg["AS"]
        s = cal.next_work(k, status + ONE)
        EF[c] = cal.shift(k, s, pg["rem"] - 1) if pg["rem"…
        continue
    lo = [start] + ([status + ONE] if status else [])
    cons = [(p, EF[p] + ONE * (1 + lag) if t == "FS" el…
    ES[c] = cal.next_work(k, max(lo + [x[1] for x in c…
    EF[c] = cal.shift(k, ES[c], a["dur"] - 1)
```

py — 계산 절차. 검증기가 FS·SS 규칙으로 날짜를 계산

핵심 모두 메모장으로 열리는 글자 파일 → AI가 읽고 고치기 쉽다. hwpx·xlsx·pptx는 압축 묶음 → 변환 필요(5주차)

A 데이터팩 한눈에 — 지하2층·지상5층 근린생활시설

파일	내용
00_README.md	프로젝트·파일·계산 규약 안내 (AI가 가장 먼저 읽음)
01_검토기준.md	A 자동 검증 / B 공정표 내용 / C 산출물 / D 사람 판단
project.json	개요·마일스톤 9개·간접비·지체상금
activities.csv · relations.csv	WBS 3단계 148개 작업 · 선후관계 272개(FS·SS·시차)
calendar.csv	날짜별 작업 가능 여부(STD·WX), 공휴일·기상
baseline_schedule.csv · schedule.json	기준 공정표(시작·종료·여유·주공정) — 그대로 그리면 됨
progress_2026-09-30.csv · crash.csv	6개월차 실적 · 단축 가능 작업과 비용
check_schedule.py	검증기 (파이썬 표준 라이브러리만)

흐름

우리의 작업 흐름 — 7단계



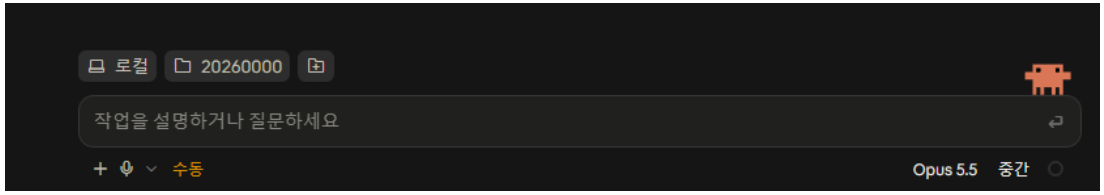
핵심

- 긴 프롬프트를 한 번에 쓰지 않는다 — 짧게 묻고, 정리하고, 합의하고, 기준을 정한 뒤 맡긴다
- AI가 스스로 고치는 동안 사람은 '이상한 수정'을 보고, 마지막 판단을 한다

흐름

① 폴더·작업 정의

[폴더 설정]



나

schedule 프로젝트 폴더를 우리 규칙대로 만들고 데이터팩 (“폴더명”)을 sources에 넣어 줘.
오늘 최종 결과물은 outputs/schedule.html이고,
형식과 계산 규칙은 sources/00_README.md를 따를 거야.
지금은 폴더만 만들고 결과물은 아직 만들지 마.

폴더명 예 : ("D:\사용자 폴더\Administrator\Downloads\데이터팩")

Claude

01_schedule/ 폴더를 규칙대로 만들었습니다(HANDOFF.md · sources · _scripts · outputs).
데이터팩 11개를 sources에 복사하고 원본과 같은지 확인했습니다.
schedule.html은 아직 만들지 않았고 outputs는 비어 있습니다.

①이 끝난 뒤 폴더

```
내 작업 폴더/
├── CLAUDE.md          ← 2주차 규칙
├── 데이터팩/          ← 내려받은 원본 (그대로 둠)
└── 01_schedule/       ← ①에서 Claude가 만든 폴더
    ├── HANDOFF.md     ← 상태·맥락·노하우
    ├── sources/       ← 데이터팩 11개 복사 (원본과 동일 확인)
    │   ├── 00_README.md · 01_검토기준.md
    │   ├── project.json · activities.csv · relations.csv
    │   ├── calendar.csv · baseline_schedule.csv · schedule.json
    │   ├── progress_2026-09-30.csv · crash.csv
    │   └── check_schedule.py
    ├── _scripts/      (비어 있음)
    └── outputs/       (비어 있음 — 아직 만들지 않음)
```

포인트

- 최종 결과물·따를 규칙을 먼저 말한다
- "아직 만들지 마" — 없으면 공정표까지 다 만들어 버린다.

흐름

② 대화로 맥락 쌓기·정리

나

baseline_schedule.csv의 주공정(★)이 이 현장에서 어디를 지나는지 한 문장으로 말해 줘.

Claude

착공신고·가설 → CIP 흙막이 → 어스앵커 터파기 → 매트기초 → 지하2층~옥탑 골조 → 외벽·조경·인입 → 사용승인·준공검사로 이어집니다.
……

나

그 근거는? 어느 파일 몇 번째 줄?

Claude

주공정(★) 세부작업: 63개
전체 세부작업: 148개
그래서 세부작업의 약 43%(63 ÷ 148)가 주공정에 올라 있습니다.
출처: baseline_schedule.csv
★ 개수와 구분=세부 행 수를 직접 세었습니다.

나

지금까지 이해한 것과 오늘 목표를 HANDOFF.md 맥락 절에 정리해 줘.

Claude

맥락 절에 '프로젝트 이해'와 '오늘 목표'를 추가했습니다. 확인 못 한 것은 [[확인필요]]로 남겼습니다.

01_schedule/HANDOFF.md — 맥락 절

프로젝트 이해 (2026-10-12 대화 정리)

- 현장: 지하2층·지상5층·옥탑, 세부작업 148개. 착공 2026-04-01, 계약 준공 2027-10-15(563일), 진도 기준일 2026-09-30. 지체상금 하루 525만원 (출처: README 1절).
- 관계: FS는 후행 시작 ≥ 선행 종료일 + 1 + 시차, SS는 후행 시작 ≥ 선행 시작일 + 시차. 그날이 후행 캘린더 휴무면 다음 작업 가능일 (README 3절).
- FS 예: 2.1.2 → 2.1.3, 시차 3, "CIP 양생" (relations.csv)
- SS 예: 3.2.1 → 3.2.2, 시차 7. 비고가 비어 7일의 근거는 파일에 없음 [[확인필요: 시차 7일 근거]]
- 시차가 달력일인 이유: 콘크리트 양생은 휴일에도 흐르기 때문. 기간은 작업일, 시차는 달력일로 구분 (README 3절 4항).
- 주공정(★): `baseline_schedule.csv`의 `주공정` 열(13번째 열)에 ★인 세부작업 63개 (요약 행 없음). 줄 번호는 헤더=1번 기준 4, 5, 14, 17~19, 22~26, 29, 30, 32, 34, 37~41, 43~47, 49~53, 55~59, 61~65, 67~71, 73~77, 79~83, 142, 144, 146~148, 178, 186, 187.
- 경로 요약: 착공신고·가설 → CIP 흙막이 → 어스앵커 2단 터파기(GL-9.2m) → 버림·방수·매트기초 → 지하2층~옥탑 층별 골조 → 외벽 마감·조경·상하수도 인입 → 사용승인 → 준공검사(8.1.6, 줄 187, 종료 2027-10-15).
- "외부 마감이 더 늦게 끝난다"는 말은 추론이며 파일에 근거 없음. 사실로 쓰지 않는다.
- 검증기 출력과의 일치는 아직 미확인 [[확인필요: 검증기 출력과 ★ 목록 일치 여부]]

포인트

- 짧게, 여러 번 묻는다
- "근거는? 몇 번째 줄?" — AI가 스스로 파악, 필요 시 정정
- 마지막엔 "정리해서 파일로" → 컨텍스트

흐름

② 내 검토기준 만들기 → 사전기준과 비교

나

오늘 만들 공정표가 잘 만들어졌는지 무엇으로 확인하면 될까?
AI가 스스로 확인할 것과 내가 직접 볼 것을 나눠서 outputs/내_검토기준.md에 정리해.
sources/01_검토기준.md는 아직 열지 말고, 공정표도 아직 만들지 마.

Claude

내_검토기준.md를 만들었습니다.
AI 스스로 확인 13개: 검증기 --compare [통과], 작업 148개·마일스톤 9개, 주공정(★) 63개 일치, sources 불변 …
직접 볼 것 8개: 한눈에 읽히는지, 작업 3~5개 표본 대조, 화면 구성 …
01_검토기준.md는 열지 않았고 공정표도 만들지 않았습니다.

나

이제 sources/01_검토기준.md(사전기준)를 읽고 내_검토기준.md와 비교해서 outputs/기준비교.md에 표로 정리해 줘.
사전기준의 항목 번호(A1, C1 같은)는 그대로 쓰고,
둘 다 있는 것·내 기준에만 있는 것, 사전기준에만 있는 것으로 나눠.
어려운 말(baseline, 검증기 등)은 쉽게 풀어 줘.
오늘 실습은 수업 사전기준으로 진행, 내 기준은 비교용이라고 HANDOFF.md에 적어 둬.

Claude

기준비교.md에 처음 보는 말 풀이와 비교표 3개를 정리했습니다.
둘 다 있는 것 13건 · 내 기준에만 8건 · 사전기준에만 6건 — 간트 눈금·진도 기준일 선(B1), 드릴다운 네트워크(B4), report.md(C3), '왜 이 순서인지' 설명(D-2) 등.
HANDOFF.md에도 적었습니다.

outputs/기준비교.md (일부)

0. 처음 보는 말 풀이

용어	쉬운 뜻
baseline(기준 공정표)	검증기가 규칙대로 미리 계산해 둔 "정답 일정". `baseline_schedule.csv`. 우리 공정표는 이 값으로 그림
검증기	`check_schedule.py`. 내가 그린 일정이 정답·규칙과 맞는지 자동으로 채점해 주는 프로그램. 마지막 줄이 [통과] / [실패]

3. 사전기준에만 있는 것

사전기준	내용	비고
B1	전체 간트: 레벨1·2 요약 막대, 월 단위 눈금, 진도 기준일(2026-09-30) 세로선	내 기준에는 구체 요구가 없었음
B4	드릴다운: 레벨2 클릭 시 ① 네트워크 다이어그램(FS/SS·시차 표기, 주공정 빨강) ② 표(기간·시작·종료·여유)	가장 큰 구현 항목
C3	`outputs/report.md`: ① 준공일·주공정 한 줄 설명 ② 검토 회차별 점검표(미달 → 수정)	내 기준에 없는 산출물
C4 산출물	`outputs/관리검토.md`(지연 일수·원인·영향 마일스톤, 단축안 평가·채택 이유), `outputs/단축안.csv`	도전. 내 A8·A9는 확인만 있고 산출물은 없었음
D-2	드릴다운 화살표 한 줄기를 따라가며 "왜 이 순서인지" 설명	사람이 이해했는지 확인
D-3	(도전) 단축안이 현장에서 가능한가(인력·공간·안전·품질), AI가 놓친 것	비용 계산 밖의 현장 판단

실습은 "사전기준"으로 진행

01_검토기준.md

- 내 기준 = 비교용 (HANDOFF.md에 기록)
- 숫자 확인은 AI도 잘 본는다
- 화면 모양·사람 판단은 사람이 정한다

흐름

③ 절차(plan.md) 정의

나

01_검토기준.md의 C1~C3를 만드는 절차를 outputs/plan.md에 10줄 이내로 써 줘.
읽을 파일, 만들 것, 확인 방법을 포함하고,
일정 값은 baseline 그대로. 아직 실행하지 마.

Claude

plan.md에 10줄로 썼습니다. 읽을 파일·만들 파일·확인 방법(A1~A3)을 넣었고, 아직 아무것도 실행하지 않았습니다.

나

네트워크는 클릭한 공종만 보이게, 외부 라이브러리 없이 SVG로.
이렇게 plan.md를 고치고 확정해.

Claude

5번을 고치고 '확정, 미실행'으로 표시했습니다.
확인 요청: 그러면 다른 공종과 이어진 관계는 네트워크에서 보이지 않습니다. '외부 연결 있음' 표시만 넣을까요?

outputs/plan.md (확정, 미실행)

C1~C3 제작 절차 (확정 2026-10-12, 미실행)

1. 읽기: `sources/` 의 project.json(마일스톤 9개·준공일), activities.csv(WBS), relations.csv(관계·시차), baseline_schedule.csv(일정·여유·★), progress_2026-09-30.csv(기준일 선). 일정은 baseline_schedule.csv 값을 그대로 쓰고 새로 계산하지 않음.
2. 스크립트: `_scripts/build_schedule.py` (표준 라이브러리만)가 위 파일을 읽어 데이터를 한 파일 안에 JSON으로 넣고 HTML을 생성 → 인터넷 없이 열림(B6).
3. C2 `outputs/일정_표시용.csv` : 세부작업별 `작업코드,시작일,종료일,여유` 를 baseline_schedule.csv에서 그대로 추출(A1 대조용).
4. C1 `outputs/schedule.html` 필수: 레벨1·2 요약 막대 간트, 월 눈금, 2026-09-30 세로선(B1) / 마일스톤 ◆ 9개 이름·날짜, 계약 목표일과 다르면 차이 표시(B2) / ★ 작업 빨강 + 범례 "준공일을 결정하는 경로"(B3).
5. C1 기본: 레벨2 공종 클릭 시 **클릭한 공종의 세부작업만** 네트워크 다이어그램(상자=작업, 화살표=관계, FS/SS·시차 표기, ★ 빨강)을 **외부 라이브러리 없이 순수 SVG**로 그리고 표(기간·시작·종료·여유)도 함께 표시(B4). 관계는 relations.csv에서 읽고, 다른 공종과 이어진 관계는 그리지 않음.
6. 검증 A1: `python3 -I sources/check_schedule.py --compare outputs/일정_표시용.csv` 가 `[통과]` .
7. 검증 A2: `python3 -I sources/check_schedule.py` 출력의 준공일·마일스톤 9개 날짜를 schedule.html 표기와 대조.
8. 검증 A3: schedule.html의 빨강 작업 목록과 baseline_schedule.csv `주공정=★` 63개를 스크립트로 비교.
9. C3 `outputs/report.md` : ① 준공일·주공정 한 줄 설명 ② 검토 회차별 점검표(미달 → 수정). 미달이 있으면 고치고 6~8을 다시 실행해 회차 기록.
10. 마무리: HANDOFF.md 상태 절 갱신, 백업 안내. 사용자 게이트(D)는 사용자가 판단. 도전 C4는 범위 확인 후 진행.

포인트

- "아직 실행하지 마" — 계획만 먼저
- 읽고 고친 뒤 "확정해"
- 학생이 정할 것: '외부 연결 표시' 넣을지

흐름

④ 검토기준·검토 방식·수정 방식 정의

나

이 공정표가 잘 만들어졌는지 무엇으로 확인하면 좋을까? 01_검토기준.md 를 바탕으로 빠진 기준이 있으면 제안해 줘.

Claude

허점: 자동 검증(A1)은 일정 숫자만 봅니다. 그림이 맞게 그려졌는지는 보장하지 않습니다.
제안: A4 요약 막대 = 자식 작업의 처음~끝 / A6 공종별 화살표 수 = relations.csv / A7 모든 공종을 눌러 오류 없이 열림 / A8 외부 인터넷 참조 0건 /
B8 빨강만으로 구분하지 않기 … (10개)

나

좋아, 제안한 것 중 꼭 필요한 것만 기준에 넣어.
검증은 반드시 검증기를 실행해서 확인해.
미달이면 고치고 다시 점검, 최대 5회. sources는 절대 고치지 마.
회차별 점검표를 report.md 끝에 남겨. 합의한 기준과 방식은 HANDOFF.md에 적어 두고, 아직 실행하지 마.

Claude

A4·A6·A7·A8을 넣고, 뺀 것은 이유와 함께 HANDOFF.md에 적었습니다.
실행 전후 sources 파일 해시를 비교하겠습니다. 아직 실행하지 않았습니다.

HANDOFF.md — 결정 기록

결정 기록

- 2026-10-12: `outputs/plan.md` 확정(C1~C3 절차 10줄). 일정 값은 baseline_schedule.csv 그대로 사용.
- 2026-10-12: 검토 기준·방식 합의 (실행 전). `sources/01_검토기준.md`는 수정 금지라 추가 기준은 이 절과 `outputs/plan.md`에 둔다.
- 기준: 기존 A1~A3(검증기) + 추가 A4 요약 막대=자식 최소시작~최대종료 / A6 공종별 화살표 수=relations.csv 내부 관계 수 / A7 모든 레벨2 공종 클릭 시 오류 없음(브라우저) / A8 외부 URL 0건. 기존 B1~B6, C1~C3, D 유지.
- 제외(필요 시 나중에): A5 작업 수 일치, B7 휴무·라벨 표본 대조, B8 색 외 구분, B9 캡처 확인, B10 외부 연결 표시. 이유: 사용자가 "꼭 필요한 것만" 지시.
- 방식: A는 반드시 검증기 실제 실행. 미달이면 수정 후 재점검, 최대 5회(5회째도 미달이면 중단·보고). 회차별 점검표는 `report.md` 맨 끝.
- 규칙: `sources/` 절대 수정 금지(실행 전후 해시 비교). 브라우저 확인이 불가하면 A7은 통과로 쓰지 않고 "미확인". 상태는 아직 미실행.

포인트

- 기준은 AI가 제안 → 사람이 고른다
- A4~A8 = 이번 대화에서 추가한 기준 (원본 A1~A3는 그대로)
- 합의 기준은 HANDOFF.md에 — sources 원본은 고치지 않는다
- 할 일: AI가 고른 기준을 읽고 넣을지·뺄지 확정

흐름

⑤ 실행

나

plan.md대로 진행해. 단계마다 무엇을 만들었는지 한 줄로 보고해.

Claude

build_schedule.py → 일정_표시용.csv(148행) → schedule.html(간트 ·◆ 9개·빨강·드릴다운) → 검증기 A1 [통과] … 3회차에 전부 통과.
질문: 층을 건너뛰는 우회선도 빨강으로 보입니다. 실제 주공정 연결만 빨강으로 줄일까요?

문제가 있을 경우,

나

(html을 열어 보고) 4.4 2층 골조를 눌렀더니 화살표 들이 박스에 가려지거나 겹쳐 보여서 구분이 잘 안된다.

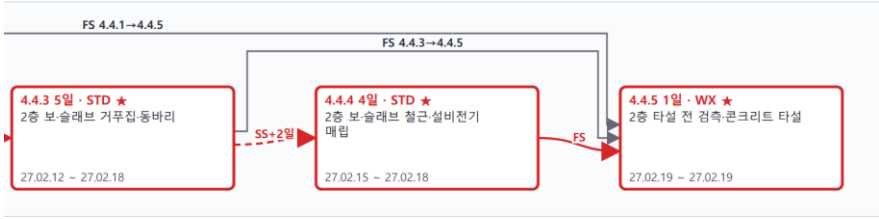
그리고 빨강 화살표는 실제 주공정 연결만 빨강으로 굵게 표현해 줘.

Claude

두 선이 같은 높이로 돌아 같은 자리에 도착해서였습니다. 선마다 높이를 따로 쓰고 'FS 4.4.1→4.4.5'처럼 출발·도착을 적었습니다. 빨강은 실제 주공정 연결만 남겼습니다.



지적 전 — 위로 도는 FS 두 개가 겹침 (4.4.3~4.4.5 확대)



지적 후 — 출발·도착 표기, 빨강은 실제 연결만

포인트

- 권한 창 = 도구 사용 요청 → 읽고 승인
- 중간 결과를 열어 보고 바로 말한다

흐름

⑥ 자동 피드백 — 검토 루프

나

검토기준과 방식대로 스스로 점검하고, 전부 통과할 때까지 반복해.

Claude

1회차: 검증기 항목 모두 통과. 글자 겹침을 화면 좌표로 재니 36건 실패 — 눈으로는 못 찾은 결함
(라벨 1px 잘림 18건, 마일스톤 선이 날짜 글자를 지남 9건) → 수정
2회차: 전부 통과. sources 해시 11개 동일.

나

까다로운 공정관리 실무자 역할의 서브에이전트에게 schedule.html과 report.md를 검토시켜서 지적사항만 받아 와.

Claude

지적 15건(치명 2·중요 8·사소 5).
예: 진도 파일은 이미 지연을 보여 주는데 화면은 '차이 0일'로 단정 /
★ 작업에도 여유 1~4일이 있는데 설명이 없음. 수정은 하지 않았습니다.

outputs/report.md — 회차별 점검표(6~7회차)

회차	기준	결과	미달 내용 → 수정 내용
6	(사용자 재지시: 합의한 기준으로 처음부터 전부 다시 점검, 새 점검 주기 1회차) A1 --compare, A2~A4, A6~A10, sources 불변	통과	이전 결과를 재사용하지 않고 전부 재실행해 통과
6	A11(B6의 자동 측정, 브라우저에서 글자 상자 좌표를 읽어 겹침·화면 밖을 측정)	실패	36건. ① 우회선 라벨이 위로 1px 잘림 18건(18개 공종) → 위쪽 여백·레인 시작 높이 확대 ② 마일스톤 선이 자기 날짜 글자를 지남 9건 → 글자를 선 옆으로 옮기고 선은 글자 윗단부터 시작 ③ 마일스톤 글자끼리 겹침 9건 → 한 마일스톤의 이름·날짜 두 줄이 맞닿은 것을 센 측정 오탐이라 점검 코드에서 같은 마일스톤끼리는 제외(제품 수정 없음)
7	A1 --compare, A2~A4, A6~A11, sources 불변	통과	전 항목 통과. A11 겹침 0건(공종 30개 + 간트). A1은 148개 세부작업 일정 일치
7	B1~B6 (최종 빌드 화면: 간트 상단, 드릴다운 3.2·4.4·7.5)	통과	B1 월 눈금·진도 기준일 선·대공종/공종 요약 막대, B2 마일스톤 ◆ 9개 이름·날짜(계약 목표일과 차이 0이라 차이 표기 없음, 표에 차이 열 있음), B3 빨강+범례 "준공일을 결정하는 경로", B4 클릭한 공종만 네트워크(FS/SS·시차, 빨강)와 표, B5 값은 모두 데이터팩에서 읽음, B6 겹침·잘림 없음

포인트

- 눈대중 → 실측으로 바꾸면 더 찾는다
- 검토자를 바꾸면 또 나온다
- → 루프 엔지니어링

흐름

⑦ 최종 사람 검토

나

crash.csv를 보고 계약 준공일을 지키는 단축안을 만들어 --crash로 평가해 줘. 결과는 outputs/관리검토.md에.

Claude

권장안: 지하·지상 거푸집 4개 작업, 추가 700만원 → 준공 2027-10-07(계약 −8일), 순효과 +8,000만원.
참고: 지하2층 거푸집 하루(140만원)만 줄여도 12일이 당겨지지만, 그 작업 하나가 안 되면 +12일로 돌아갑니다.

나

지하 거푸집은 CIP 합벽이라 작업 공간이 좁고 겨울철이라 증원이 어려워. 지하층은 빼고 지상층 골조 위주로 계약 준공을 지키는 안이 있는지 --crash로 확인해 줘. 숫자는 검증기 출력으로만 말해.

Claude

있습니다. 3·4·5층 거푸집 각 1일(420만원) → 2027-10-15(계약 0일).
단, 하나만 빠져도 +5일.
비계 해체 2일·옥탑 거푸집 1일을 더하면(880만원) 2027-10-08(−7일)이고, 한 작업이 빠져도 계약 준공을 지킵니다.

outputs/관리검토.md — 지하를 뺀 단축안(검증기 평가)

8.1 안별 검증기 평가

안	단축 작업(일)	추가비용(만원)	준공(검증기)	순효과(만원)	한 작업이 빠지면(검증기 실측)
골조만(최소비용) 단축안_지상층_골조만_2026-10-12.csv	4.5.2(1), 4.6.2(1), 4.7.2(1)	420	2027-10-15 (계약 대비 +0일)	+7,320	셋 중 **어느 하나만 빠져도 2027-10-20(+5일)**
불발 내성 단축안_지상층_불발내성_2026-10-12.csv	1.2.5(2), 4.5.2(1), 4.6.2(1), 4.7.2(1), 4.8.2(1)	880	2027-10-08 (-7일)	+7,700	1.2.5가 빠지면 10-13(-2일), 4.5.2·4.6.2·4.7.2가 빠지면 10-15(+0일), 4.8.2가 빠지면 10-13(-2일). 1.2.5를 1일만 줄이면 10-12(-3일)

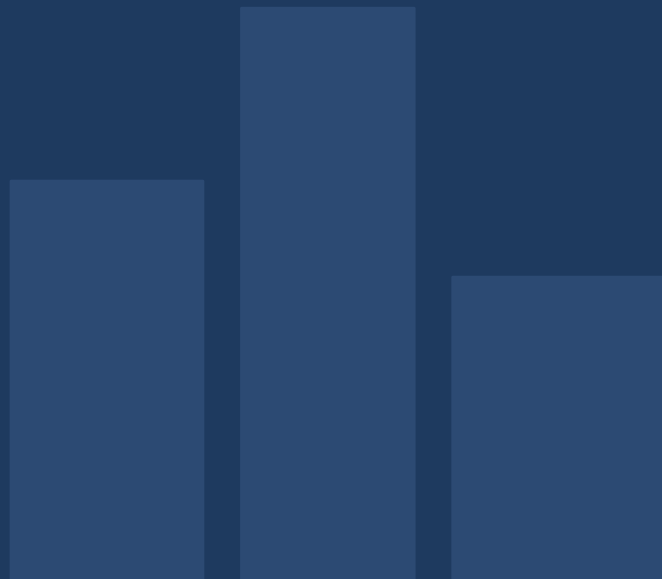
모두 할 일 / 위 대화 = 도전 예시

- 모두: ◆ 3개 날짜 대조 → 읽기 어려운 곳 재지시
- 도전: 진도 반영·단축안 (위 대화)
- AI는 현장 조건을 모른다 → 사람이 준다
- "숫자는 검증기 출력으로만"

Part B

공정표로 관리하기

계산은 AI가, 읽기와 판단은 사람이



B 공정표란 — 설계 일정표의 공사판

공정관리 = 일정·자원·비용을 통합적으로 계획·통제하는 활동

설계 일정표

계획설계 → 중간보고 ◆ → 기본설계 → 실시설계 → 최종납품 ◆

공사 공정표

착공 ◆ → 흙막이·토공 → 기초·골조 → 마감·설비 → 사용승인·준공 ◆

다른 점

■ 작업 수가 수백 개 (오늘 데이터 148개) · 작업끼리 순서가 얽혀 있다 · 날씨·휴일이 일정을 바꾼다

핵심 마일스톤(◆) = 반드시 지켜야 하는 중간 약속 날짜

B 공기는 어떻게 정해지나 — 국토교통부 고시

「공공 건설공사의 공사기간 산정기준」(국토교통부 고시) 제6조

준비기간	작업일수	비작업일수	정리기간

구성	고시 내용	오늘 데이터팩에서
준비기간	측량·가설·자재 조달 등 착공 준비	1.1 착공 준비 작업
비작업일수	법정공휴일 + 폭염·폭우·혹한 등 날씨로 못 하는 날	calendar.csv — STD 표준 달력(주말·공휴일 휴무), WX 기상 달력(+비·폭염 등)
작업일수	작업량 기준 또는 과거 실적·경험치	activities.csv 기간(작업일)
정리기간	준공검사 준비·청소 등	8.1.6 준공 청소·준공검사·인수

핵심 오늘 현장: 착공 2026-04-01 → 계약 준공 2027-10-15 (563일)

B WBS = 공사의 목차



WBS 읽는 법

- 작업코드 4.4.5
  = 4 골조 → 4.4 2층 골조
  → 5번째 세부작업
- 상위작업의 기간
  = 하위작업들 중 가장 먼저 시작하는 날~
  가장 늦게 끝나는 날까지

오늘 데이터 (activities.csv)

- L1 대공종 8
  → L2 공종 30
  → L3 세부작업 148
- 작업코드 1 / 1.1 / 1.1.1 = 목차 번호

핵심 WBS = 프로젝트를 관리 가능한 작업 단위로 계층 분해한 목록

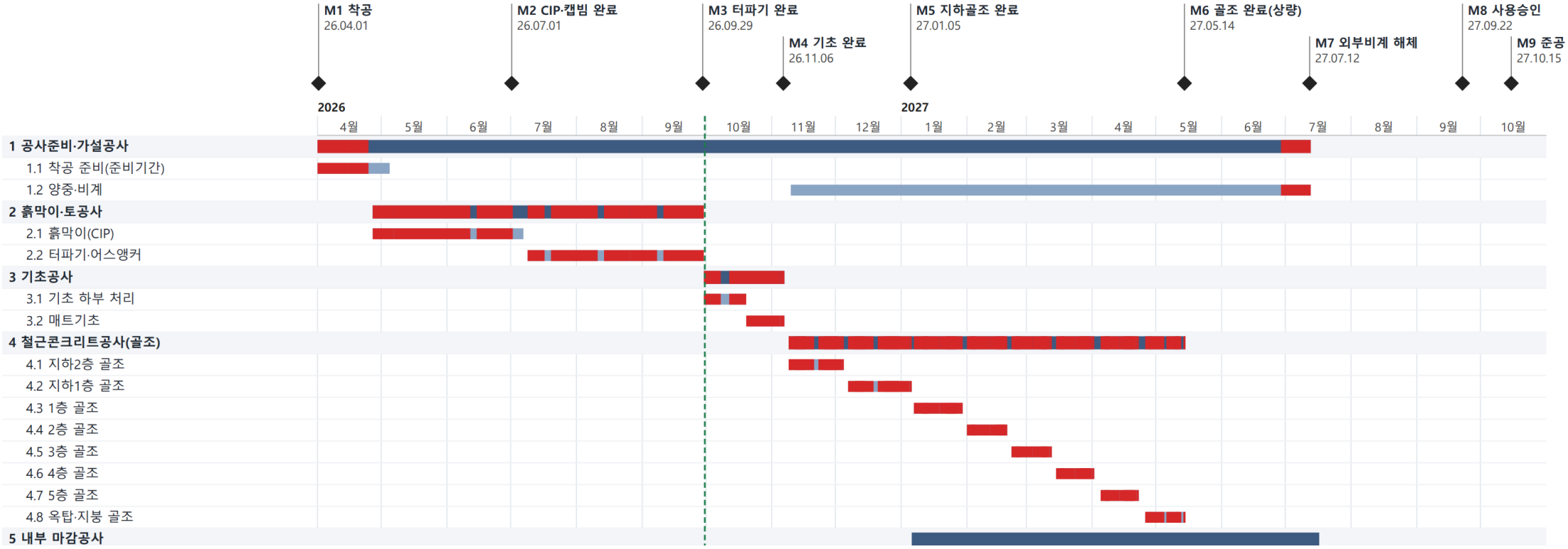
B

간트차트와 마일스톤 읽기

창원 ○○동 근린생활시설 신축공사(교육용 가상 현장) - 공정표

착공 2026-04-01 → 준공 2027-10-15 (공기 563일, 계약 준공 2027-10-15) · 진도 기준일 2026-09-30 · 일정 출처: 기준 공정표(baseline_schedule.csv)

주공정 = 준공일을 결정하는 경로 | 일반 작업 구간(요약 막대) | ◆ 마일스톤(주요 일정 시점) | 진도 기준일 | ※ 공종(레벨2) 행을 클릭하면 아래에 세부 네트워크와 표가 열립니다.

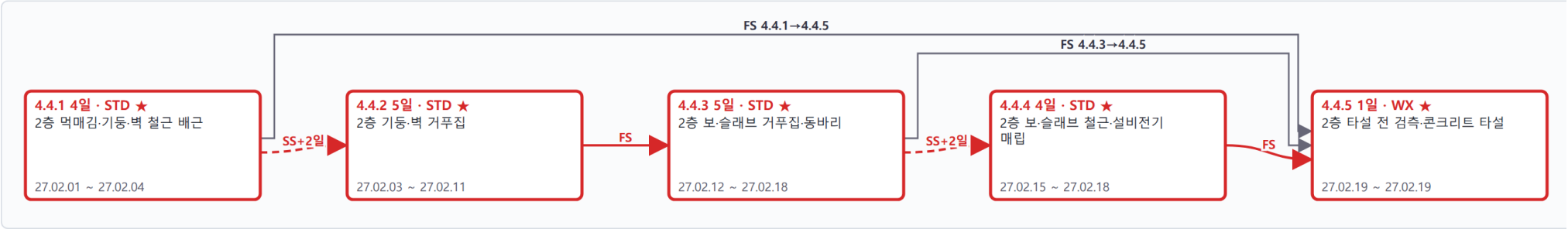


막대 기간 **겹침** 동시 진행 ◆ 마일스톤 **빨강** 주공정 **점선** 진도 기준일(9/30)

따라 읽기

[◆ M2 CIP·캡빔 완료 = 26.07.01] [2.1 흙막이(CIP) 막대 = 4월 말~7월 초] [빨간 막대 = 주공정, 초록 점선 = 9/30]

B 순서와 네트워크 — 골조 한 층의 사이클





- FS (실선): 앞 작업이 끝나면 시작
- SS +2일 (점선): 앞 작업 시작 2일 뒤 시작 — 겹쳐 진행
- 시차 = 달력일: 양생처럼 휴일에도 흐르는 시간

골조 한 층의 사이클 (매 층 반복)

기둥·벽 철근 → 기둥·벽 거푸집 → 보·슬래브 거푸집 → 슬래브 철근·매립 → 검측·타설

B 주공정 = 준공을 결정하는 길

1. 화살표를 따라가는 길 중 가장 긴 길 = 공기
2. 그 길 = 주공정 (빨강)
3. 빨간 길 위가 늦으면 → 준공이 늦어질 수 있다
4. 빨간 길 밖은 → 여유만큼 늦어도 괜찮다

계산은 누가?

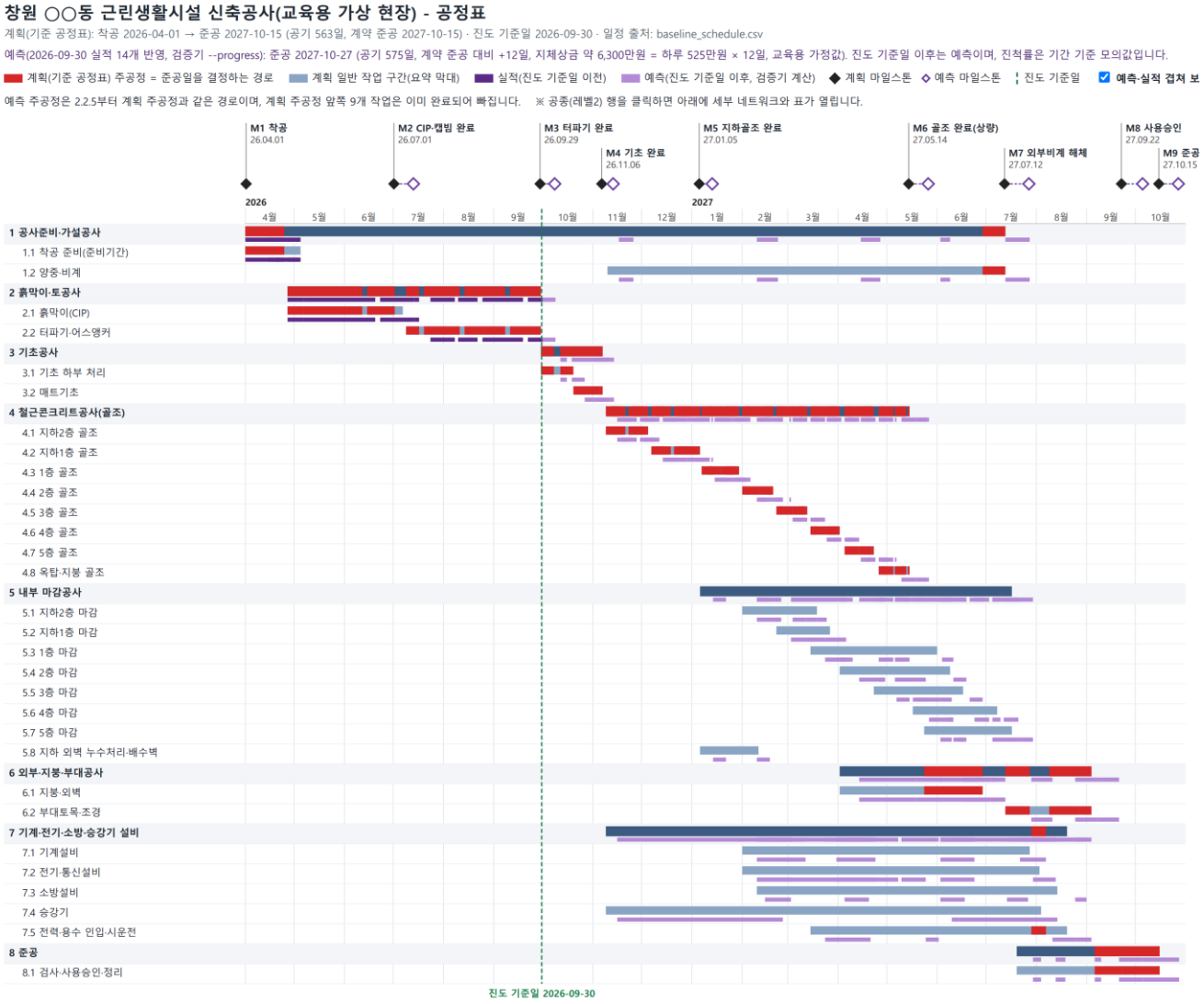
- AI가 데이터로 그리고
- 검증기가 따로 계산해 대조
- 사람은 빨간 길을 읽고 판단

오늘 현장의 빨간 길

흙막이·토공 → 기초 → 골조 8개 층 → 외벽(고층부) → 비계 해체·부대 → 사용승인·준공

참고 주말·휴일·기상 휴무가 섞이면 빨간 길 위에도 여유 1~4일이 보일 수 있다 → 판단은 ★ 표시와 검증기로

B 공정표로 관리하기 ① — 6개월차, 실적이 올라왔다



마일스톤	계획	예측	차이
CIP·캡빔 완료	26.07.01	26.07.13	+12일
터파기 완료	26.09.29	26.10.08	+9일
기초 완료	26.11.06	26.11.13	+7일
골조 완료(상량)	27.05.14	27.05.26	+12일
사용승인	27.09.22	27.10.05	+13일
준공	27.10.15	27.10.27	+12일

지연 원인:
① CIP 중 미확인 도시가스관 이설
② 7월 집중호우(캡빔)
③ 2단 앵커 재천공

결과 예측 준공 +12일 → 지체상금 약 6,300만원 (하루 525만원). 공정표 = 실적을 넣어 미래를 다시 보는 도구

B 공정표로 관리하기 ② — 어디를, 얼마에 줄일까

단축 비용(인원·장비) ↔ 절감: 간접비 120만원/일 + 계약일 넘긴 날 줄이면 지체상금 525만원/일 (교육용 가정)

빨간 길 밖을 줄이면

- 비주공정 47개 작업을 최대한 단축
- 추가 1억 80만원 → 0일 당김
- 순효과 −1억 80만원 (돈만 쓴다)
- 여유가 단축 한도보다 크기 때문

AI가 찾은 최저비용안

- 지하2층 거푸집 1일만 단축 (140만원)
- → 12일 당김, 계약 준공 회복, 순효과 +7,600만원
- 이유: 목요일에 끝나면 다음 작업이 주말 전에 시작 → 층마다 누적 (달력 효과)
- 그 작업이 안 되면 다시 +12일

사람이 조건을 준 뒤

- "지하는 합벽·겨울철이라 증원 곤란"
- → 3·4·5층 거푸집 각 1일 (420만원): 계약 0일, 하나 빠지면 +5일
- → 비계 해체 2일 + 옥탑 거푸집 1일 추가 (총 880만원): −7일, 하나 빠져도 계약 준수

핵심 어디를 줄이느냐가 전부. 조합 탐색과 숫자는 AI·검증기가, 현장에서 가능한지는 사람이 판단

B AI 계산을 어떻게 믿나 — 독립 검증기

프로젝트 정보 → AI 생성 → 전문가 검토 → **최종**

MDPI 연구 (2023)

- "논리적이고 일관된 작업 분해 가능"
- 초기 정보가 적어도 구조화 능력 긍정적
- 단, 매번 다른 결과·현장 맥락 부족

ASCE 연구 결론 (2025)

- "AI 권고는 이론적·교과서적 수준"
- 폭(breadth)은 AI가
- 깊이(depth)와 실무 해석은 사람이

결론: AI = 빠른 초안 도구 + 전문가 검증 필수

검증을 실행 가능하게

```
〉python check_schedule.py --compare
   outputs\일정_표시용.csv

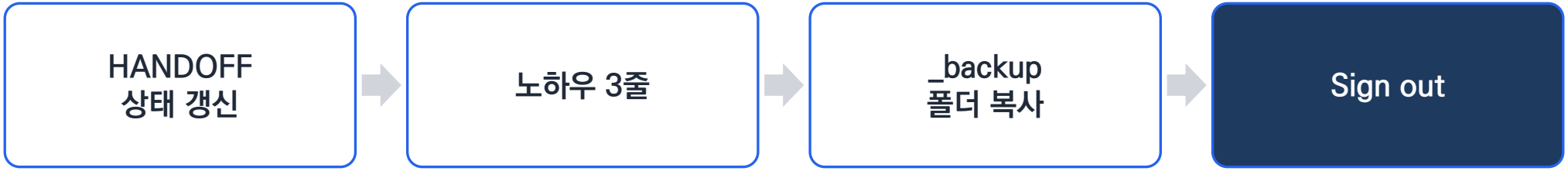
[대조] 제출 148개 세부작업 일정이
       검증기와 일치
==== 검증 결과 ====
[통과] 실패 항목 없음
```

- ① AI가 내보낸 일정_표시용.csv 날짜를 따로 계산해 대조
- ② 화면 표기는 "검토기준 B"로 점검
- ③ 현장에 맞는지는 사람이 통과
  = 합의한 항목 통과 (현장 적합성 보증 아님)

핵심 AI가 만든 숫자는 AI가 아닌 독립된 계산기로 대조한다 — 이것이 ⑥ 자동 피드백(검토 루프)의 핵심

B

마무리 루틴 — 매 회차 같은 끝



오늘 익힌 것

- 프롬프트 = 짧은 질문·지시
- 컨텍스트 = 데이터팩+HANDOFF
- 하네스 = 폴더 규칙·검토기준·검증기
- 루프 = 통과까지 반복·최대 5회

오늘의 핵심

- 긴 프롬프트 대신 7단계 흐름:

  폴더·작업 정의 → 맥락 쌓기 → 절차 → 검토기준 → 실행 → 자동 피드백 → 사람 검토
- 공정표 = 그리고 끝이 아니라, 실적을 넣어 미래를 다시 보는 관리 도구
- 단축 후보는 빨간 길에서 먼저 찾고, 효과는 검증기로 확인